

131. 分割回文串

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Medium	(Not Available)	(Not Available)	-	-	0

- **Tags:** 字符串, 动态规划, 回溯
- **Companies:** (Not Available)

给你一个字符串 `s`，请你将 `s` 分割成一些子串，使每个子串都是回文串。返回 `s` 所有可能的分割方案。

回文串是正着读和反着读都一样的字符串。

示例 1:

输入: `s = "aab"`

输出: `[["a","a","b"],["aa","b"]]`

示例 2:

输入: `s = "a"`

输出: `[["a"]]`

提示:

- `1 <= s.length <= 16`
- `s` 仅由小写英文字母组成

[Discussion](#) | [Solution](#)

解决方法

1. 问题理解

目标: 将给定的字符串 `s` 分割成若干个子串, 要求每个子串都是回文串。找出所有可能的分割方案。输入: 字符串 `s`。输出: 一个列表, 其中每个元素是 `s` 的一种有效分割方案 (方案本身是一个包含回文子串列表)。

例如: 对于 “aab” - 分割成 “a”, “a”, “b”。这三个都是回文串。方案: `[“a”, “a”, “b”]` - 分割成 “aa”, “b”。这两个都是回文串。方案: `[“aa”, “b”]`

2. 思路分析

这又是一个典型的回溯问题, 涉及到对字符串进行分割, 并检查子串的属性 (是否为回文)。

决策树模型：- 树的每个节点代表一种部分分割方案。- 从一个节点出发，尝试在剩余未分割的字符串中找到一个回文前缀，将其作为下一个分割出的子串。

回溯过程：

1. 路径 (**Path**)：记录当前已经分割出的回文子串列表 `current_partition`。
2. 选择列表 (**Choices**)：在当前位置（由起始索引 `start_index` 标记），可以尝试将 `s[start_index...i]`（其中 `i` 从 `start_index` 到 `len(s)-1`）作为下一个子串进行分割，前提是这个子串必须是回文串。
3. 结束条件 (**End Condition**)：当 `start_index` 到达字符串末尾 (`start_index == len(s)`) 时，说明整个字符串已经被成功分割成若干回文子串，找到了一个完整的分割方案。

具体步骤：

1. 回文判断函数：需要一个辅助函数 `is_palindrome(sub)` 来判断一个字符串 `sub` 是否是回文串。简单的方法是比较字符串与其反转是否相等 (`sub == sub[::-1]`)。
2. 定义回溯函数 `backtrack(start_index, current_partition)`：
 - `start_index`: 当前考虑的子串的起始索引。
 - `current_partition`: 当前已经分割出的回文子串列表。
3. 递归终止条件：
 - 如果 `start_index == len(s)`:
 - 说明已经成功分割完整字符串。将 `current_partition` 的副本加入结果列表 `result`。
 - 返回。
4. 遍历选择列表 (尝试分割)：
 - 从 `end_index = start_index` 开始，遍历到 `len(s) - 1`。
 - 取出当前尝试分割的子串 `substring = s[start_index : end_index + 1]`。
 - 检查是否为回文：调用 `is_palindrome(substring)`。
 - 如果 `substring` 是回文串：
 - 做选择：将 `substring` 加入 `current_partition`。
 - 递归进入下一层决策：调用 `backtrack(end_index + 1, current_partition)`。注意，下一个起始索引是 `end_index + 1`。
 - 撤销选择 (回溯)：将 `substring` 从 `current_partition` 末尾移除 (`current_partition.pop()`)。
5. 主函数调用：
 - 初始化结果列表 `result = []`。
 - 调用 `backtrack(0, [])` 开始回溯。
 - 返回 `result`。

优化：预计算回文子串

频繁调用 `is_palindrome` 可能会比较耗时。我们可以预先计算出字符串 `s` 中所有可能的子串是否为回文串，并存储在一个二维 DP 数组 `dp` 中。`dp[i][j]` 为 `True` 表示子串 `s[i...j]` 是回文串。

- **DP 状态定义:** $dp[i][j]$ 表示 $s[i...j]$ 是否为回文串。
- **DP 状态转移方程:**
 - $dp[i][j] = (s[i] == s[j]) \text{ and } dp[i+1][j-1]$
- **Base Case:**
 - $dp[i][i] = \text{True}$ (单个字符是回文)
 - $dp[i][i+1] = (s[i] == s[i+1])$ (两个字符的情况)
- **计算顺序:** 需要根据子串长度从小到大计算, 或者根据起始、结束索引的相对关系计算 (例如, 外层循环 j 从 0 到 $n-1$, 内层循环 i 从 0 到 j)。

在回溯过程中, 直接查询 $dp[start_index][end_index]$ 即可判断子串是否为回文, 时间复杂度为 $O(1)$ 。预计算 DP 数组的时间复杂度为 $O(N^2)$, 空间复杂度为 $O(N^2)$, 其中 N 是字符串长度。

3. 代码实现 (Python)

```
from typing import List

class Solution:
    def partition(self, s: str) -> List[List[str]]:
        """
        使用回溯算法找出所有回文分割方案。
        Args:
            s: 输入字符串。
        Returns:
            包含所有有效分割方案的列表。
        """
        n = len(s)
        result = [] # 存储最终结果
        path = []   # 存储当前分割路径

        # --- 优化: 预计算回文子串 ---
        # dp[i][j] 表示 s[i..j] 是否是回文串
        dp = [[False] * n for _ in range(n)]
        for length in range(1, n + 1): # 子串长度
            for i in range(n - length + 1):
                j = i + length - 1 # 子串结束索引
                if length == 1:
                    dp[i][j] = True
                elif length == 2:
                    dp[i][j] = (s[i] == s[j])
                else:
                    dp[i][j] = (s[i] == s[j]) and dp[i + 1][j - 1]

        # --- 回文判断函数 (使用预计算结果) ---
        def is_palindrome(start: int, end: int) -> bool:
            return dp[start][end]
```

```

# --- 回溯函数 ---
def backtrack(start_index: int):
    """
    回溯函数核心逻辑。
    Args:
        start_index: 当前考虑的子串的起始索引。
    """
    # 递归终止条件: 当起始索引到达字符串末尾时, 找到一个完整分割方案
    if start_index == n:
        result.append(path[:]) # 添加路径副本
        return

    # 遍历所有可能的结束索引 end_index
    for end_index in range(start_index, n):
        # 检查 s[start_index...end_index] 是否为回文
        if is_palindrome(start_index, end_index):
            substring = s[start_index : end_index + 1]

            # --- 做选择 ---
            path.append(substring)

            # --- 递归进入下一层决策 ---
            # 下一个起始索引是 end_index + 1
            backtrack(end_index + 1)

            # --- 撤销选择 (回溯) ---
            path.pop()

    # 从索引 0 和空路径开始回溯
    backtrack(0)
    return result

# 示例调用
# sol = Solution()
# print(sol.partition("aab"))
# print(sol.partition("a"))

```

4. 复杂度分析

设字符串 s 的长度为 N 。

- 时间复杂度: $O(N * 2^N)$
 - 预计算 DP 数组需要 $O(N^2)$ 时间。
 - 回溯部分: 状态树的节点数最多有 2^N 种可能的分割方式 (考虑每个字符间是否分割)。在每个节点处, 我们需要 $O(N)$ 的时间来遍历可能的下

一个分割点，并进行 $O(1)$ 的回文判断（使用 DP 数组）。生成最终结果时复制路径也需要时间。总的时间复杂度主要由回溯搜索决定，大致为 $O(N * 2^N)$ 。

- 空间复杂度： $O(N^2)$
 - DP 数组需要 $O(N^2)$ 空间。
 - 递归调用栈的深度最多为 N 。
 - `path` 列表最多存储 N 个字符（如果每个字符都单独分割），空间为 $O(N)$ 。
 - 结果列表 `result` 的空间取决于解的数量和长度，不计入算法本身空间复杂度。
 - 因此，主要空间开销来自 DP 数组，为 $O(N^2)$ 。如果 N 很大，可以考虑不预计算 DP，每次 $O(N)$ 判断回文，空间复杂度降为 $O(N)$ ，但时间复杂度会更高。

5. 如何在面试中讲解

1. 问题理解：
 - “目标是将一个字符串分割成多个子串，使得每个子串都是回文串，并找出所有可能的分割方案。”
2. 核心思路：回溯：
 - “这是一个典型的分割问题，可以用回溯法解决。我们尝试在字符串的不同位置进行切割，并检查切割出的子串是否是回文。”
 - “定义回溯函数 `backtrack(startIndex, currentPath)`, `startIndex` 表示当前从未分割部分的哪个索引开始，`currentPath` 存储已分割出的回文子串。”
3. 回溯过程：
 - “结束条件：当 `startIndex` 到达字符串末尾时，表示整个字符串已被成功分割，将 `currentPath` 的副本加入结果集。”
 - “选择与递归：遍历从 `startIndex` 到字符串末尾的所有可能的结束位置 `endIndex`。”
 - “取出子串 `s[startIndex...endIndex]`。”
 - “判断这个子串是否是回文串。”
 - “如果是回文串：将其加入 `currentPath`，然后递归调用 `backtrack(endIndex + 1, currentPath)` 来处理剩余部分。”
 - “递归返回后，将子串从 `currentPath` 中移除（撤销选择）。”
4. 回文判断优化：
 - “在回溯过程中，会频繁判断子串是否为回文。为了提高效率，可以预先计算并存储所有子串的回文信息。”
 - “可以使用动态规划：定义 `dp[i][j]` 表示 `s[i...j]` 是否为回文。通过 `dp[i][j] = (s[i] == s[j]) and dp[i+1][j-1]` 进行计算。预计算时间 $O(N^2)$ ，空间 $O(N^2)$ 。”
 - “之后在回溯中，判断回文只需 $O(1)$ 的查表时间。”
5. 初始调用：
 - “主函数初始化结果列表，（可选地进行 DP 预计算），然后调用 `backtrack(0, [])`。”
6. 复杂度分析：

- “使用 DP 优化后：时间复杂度大致为 $O(N * 2^N)$ （回溯部分主导），空间复杂度为 $O(N^2)$ （DP 数组主导）。”
- “不使用 DP 优化：时间复杂度更高，空间复杂度为 $O(N)$ （递归栈主导）。”